It is common to use subclassing to add application-specific behavior to a standard library or framework class. If you've used Ruby on Rails,[1] you'll have subclassed ActionController::Base when writing your own controller classes. Your controllers get all the behavior of the base controller and add their own specific handlers to individual user actions.

Let's look at an example where inheritance can spare us a significant amount of duplication. Imagine you are working on a task-tracker application. A task might be in one of several states—it might be done, it might be started but incomplete, or it might be defined but not started. There may be other statuses, but just those three are probably enough to make the point.

If you are writing code that interacts with the tasks in this system, you'll likely have to take a task's status into account in your code. In other words, you'll likely be forever writing code like this:

```
def chatty_string(task)
  case task.status
  when "done" then "I'm done"
  when "started" then "I'm not done"
  when "unstarted" then "I haven't even started"
  end
end
```

You will be continually switching on the status of a task. This is a form of duplication—if the list of statuses changes, every one of these if statements or case statements would need to be updated; therefore, it seems worth trying to reduce the amount of times we use that switching logic.

We can use inheritance to create a hierarchy of status classes, and then only do our switching logic once:

tut_modules/status.rb

```
class Status
  def self.for(status_string)
    case status_string
    when "done" then DoneStatus.new
    when "started" then StartedStatus.new
    when "defined" then DefinedStatus.new
    end
  end

  def done? = false

  def chatty_string = raise NotImplementedError
end

class DoneStatus < Status
  def to_s = "done"

  def done? = true

  def chatty_string = "I'm done"
end

class StartedStatus < Status
  def to_s = "started"
```

1. http://www.rubyonrails.org